

SI3003 - Inteligencia Artificial

CLASE 5 – APRENDIZAJE POR REFUERZO

Reinforcement Learning

El Aprendizaje por Refuerzo (**RL**) es una técnica de inteligencia artificial en la que un agente aprende a tomar decisiones mediante prueba y error, recibiendo recompensas o penalizaciones según sus acciones.

Analogía

Es como aprender a jugar un videojuego: al principio cometemos muchos errores, pero con cada intento aprendemos qué acciones nos acercan más a obtener una buena puntuación.

class: middle, center, smaller



Vamos a profundizar en uno de los métodos de aprendizaje por refuerzo, los **métodos basados en el valor**, y a estudiar un algoritmo de aprendizaje por refuerzo: el **Q-Learning**.

Hoy

La idea de esta clase es profundizar en los siguientes temas:

- Métodos basados en valores.
- Diferencias entre el aprendizaje de Monte Carlo y el aprendizaje por diferencias temporales.
- Estudiaremos e implementaremos un algoritmo de aprendizaje por refuerzo (RL): el aprendizaje Q (Q-Learning).

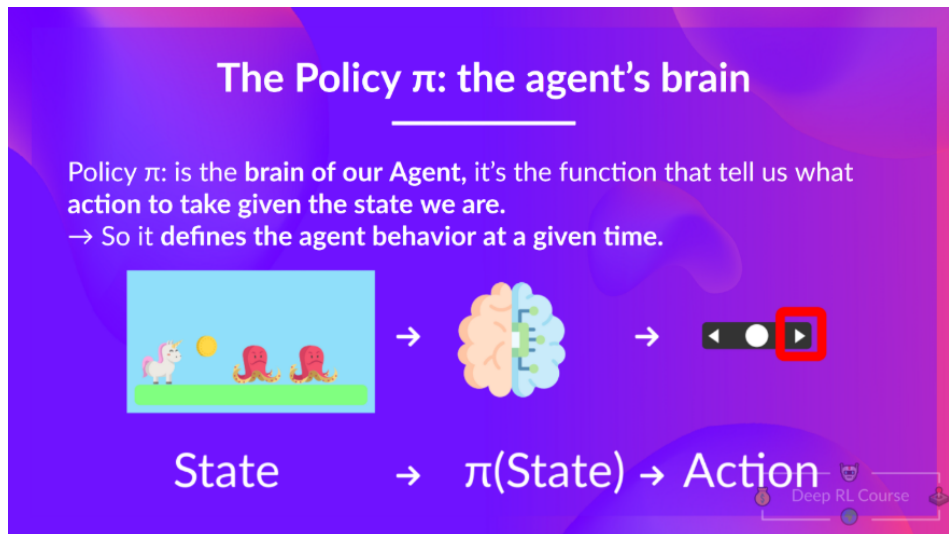
Nota: el RL es una aproximación a lo que hoy conocemos como game-playing agents.

En RL construimos agentes que toman decisiones inteligentes, por ejemplo un agente que juega videojuegos o invierte en la bolsa. Siempre buscando **maximizar sus beneficios**.



Para tomar decisiones inteligentes, nuestro agente aprenderá del entorno interactuando con él mediante el método de prueba y error y recibiendo recompensas (positivas o negativas) como retroalimentación.

El proceso de toma de decisiones del agente se denomina **política** π : dada una situación, una política generará una acción o una distribución de probabilidades sobre las acciones. Es decir, dada una observación del entorno, una política proporcionará una acción (o varias probabilidades para cada acción) que el agente debería llevar a cabo.



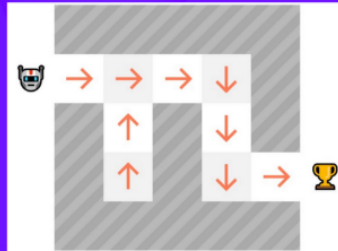
Nuestro objetivo es encontrar una **política óptima** π^* , es decir, una política que conduzca a la mejor recompensa acumulada esperada.

Y para encontrar esta política óptima (y, por lo tanto, resolver el problema de RL), existen dos tipos principales de métodos de RL:

- *Métodos basados en políticas*: entrenan directamente la política para aprender qué acción tomar dado un estado.
- *Métodos basados en valores*: entrenan una función de valor para aprender qué estado es más valioso y utilizan esta función de valor para tomar la acción que conduce a él.

Two approaches to find optimal policy π^* :

Policy-Based methods: train the agent to learn which **action to take**, given a state.



Value-Based methods: train the agent to learn which state is **more valuable** and take the action that **leads to it**.



El objetivo de un agente de RL es tener una política óptima π^* .

Para encontrar la política óptima, tenemos 2 métodos diferentes:

Métodos basados en política

Entrenamos directamente la política para seleccionar qué acción tomar dado un estado (o una distribución de probabilidad sobre las acciones en ese estado).

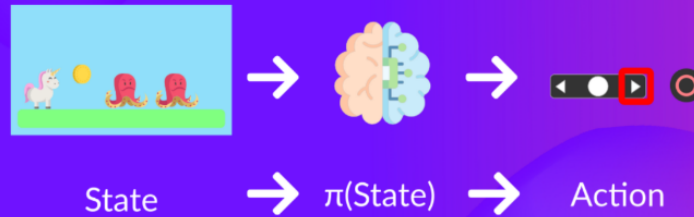
En este caso, no tenemos una función de valor. La política recibe un estado como entrada y devuelve qué acción tomar en ese estado.

*Nota: una política **determinista** es una política que devuelve una única acción dado un estado, a diferencia de una política **estocástica**, que devuelve una distribución de probabilidad sobre las acciones.*

Two approaches to find optimal policy π^* :

Policy-Based methods:

- Train directly the policy.
- Our policy is a Neural Network.
 - No value function.



No definimos manualmente el comportamiento de nuestra política; es el **entrenamiento** el que lo definirá.

Métodos basados en valores

De forma indirecta, entrenando una función de valor que devuelve el valor de un estado o de un par estado-acción. Dada esta función de valor, nuestra política tomará una acción.

Como la política no se entrena ni se aprende, tenemos que especificar su comportamiento. Por ejemplo, si queremos una política que, dada la función de valor, tome acciones que siempre conduzcan a la mayor recompensa, crearemos una **Política Greedy** (codiciosa).

Nota: greedy significa que la política elige en cada momento la acción que parece mejor según la función de valor.

Value-Based Methods

The value of a state is the **expected discounted return** the agent can get if it **starts in that state**, and then **act according to our policy**.

$$v_{\pi}(s) = \mathbb{E}_{\pi} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s]$$

Value
function

Expected discounted return

Starting
at state s

Two approaches to find optimal policy π^* :

Value-Based methods:

- Don't train the policy.
- Our policy is a function defined by hand.
- Instead train a value-function that is a Neural Network.



Dado un estado, nuestra función action-value (la cual entrenamos) genera el valor de cada acción en ese estado; luego nuestra política "*greedy*" selecciona la acción que proporcionará el mayor valor para un estado determinado.

En consecuencia, independientemente del método que utilices para resolver tu problema, tendrás una política.

Solo que en el caso de los métodos basados en valores, no entrenas la política: tu política es simplemente una función predefinida (por ejemplo, la Política Greedy) que utiliza los valores proporcionados por la función de valor para seleccionar sus acciones.

Entonces, la diferencia es:

- En el entrenamiento basado en políticas, la política óptima (denotada como π^*) se encuentra entrenando directamente la política.
- En el entrenamiento basado en valores, encontrar una función de valor óptima (denotada como Q^* o V^*) conduce a obtener una política óptima.

The link between Value and Policy:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Finding an optimal value function leads to having an optimal policy.

Encontrar una función de valor óptima lleva a tener una política óptima.

La función de valor de estado

Escribimos la función de valor de estado bajo una política π de la siguiente manera:

State Value Function:

$$\underline{V_{\pi}(s)} = \underline{\mathbf{E}_{\pi}}[\underline{G_t} | \underline{S_t = s}]$$

Value of state s Expected return If the agent starts at state s

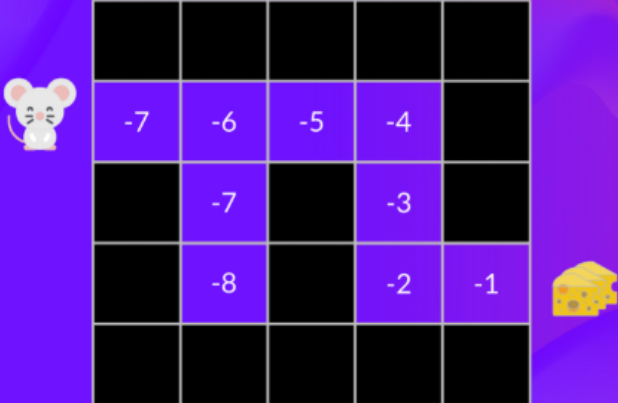
And uses the policy to choose its actions for all time steps

For each state,
the state-value function outputs
the expected return
if the agent starts in that state
and then follows the policy forever after.

Para cada estado, la función estado-valor devuelve la rentabilidad esperada si el agente parte de ese estado y, a partir de ahí, sigue la política de forma indefinida (en todos los pasos temporales futuros, si lo prefieres).

The State Value Function

State Value Function: calculate the value of a state.



A 5x5 grid world environment. A mouse icon is at the start (row 2, column 1) and a cheese icon is at the goal (row 4, column 5). The grid contains numerical values representing the state value function:

-7	-6	-5	-4	
	-7		-3	
	-8		-2	-1

La función de valor de acción

En la función de valor de acción, para cada par de estado y acción, la función de valor de acción devuelve el retorno esperado si el agente comienza en ese estado, toma esa acción y, a partir de entonces, sigue la política para siempre.

El valor de tomar la acción a en el estado s bajo una política π es:

Action Value Function:

$$Q_{\pi}(s, a) = \mathbf{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Value of state-action pair s, a Expected return If the agent starts at state s and chooses action a

And then uses the policy to choose its actions for all time steps

For each state and action, the action-value function outputs the expected return if the agent starts in that state and takes the action and then follows the policy forever after.

Action Value Function: calculate the value of state-action pair.



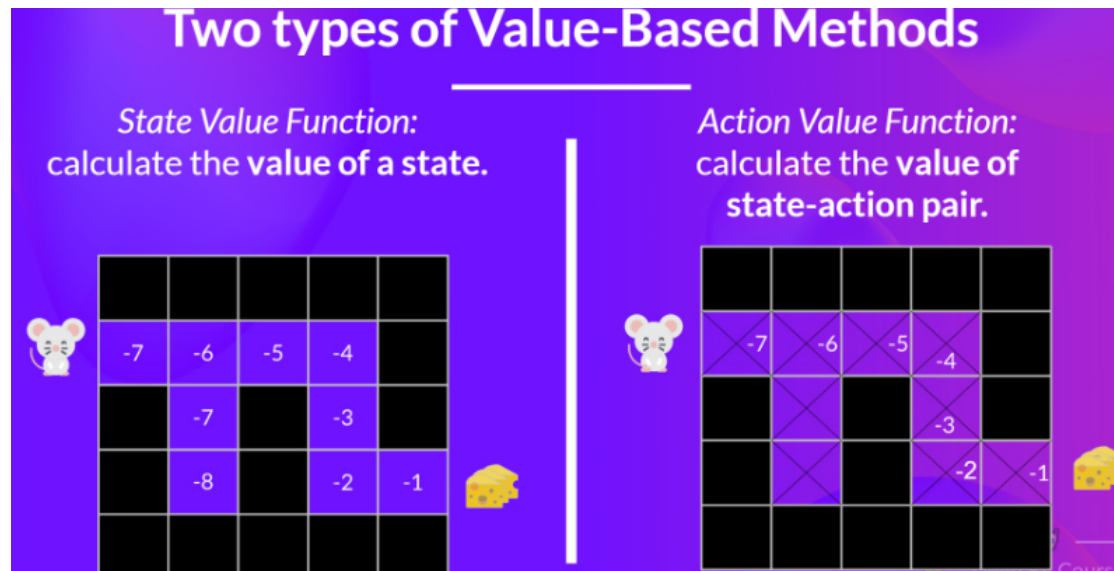
-7	-6	-5	-4	
			-3	
			-2	-1

*We didn't fill all the state-actions pair for the example of Action-value function



Para la función de valor de estado, calculamos el valor de un estado S_t .

Para la función de valor de acción, calculamos el valor del par estado-acción (S_t, A_t) , es decir, el valor de tomar esa acción en ese estado.



El problema es que para calcular cada valor de un estado o de un par estado-acción, necesitamos sumar todas las recompensas que un agente puede obtener si comienza en ese estado.

Este puede ser un proceso computacionalmente costoso, y ahí es donde la **ecuación de Bellman** entra en juego para ayudarnos.

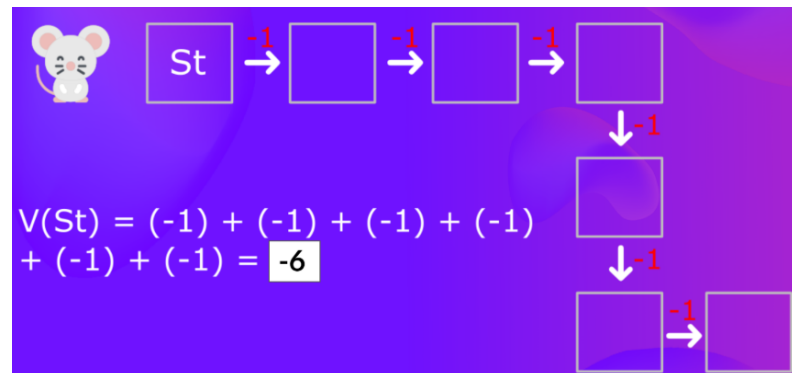
The Bellman Equation: simplify our value estimation

La ecuación de Bellman simplifica el cálculo del valor de estado o del valor de estado-acción.

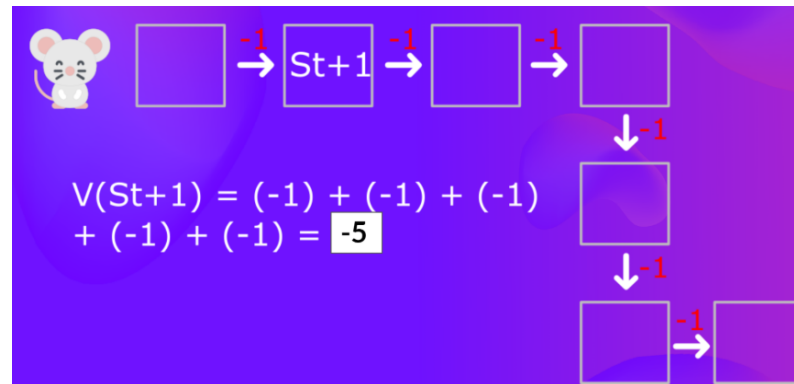
Sabemos que si calculamos $V(S_t)$ (el valor de un estado), necesitamos calcular el retorno comenzando desde ese estado y luego seguir la política para siempre.

Para calcular $V(S_t)$, necesitamos calcular la suma de las recompensas esperadas. Por lo tanto:

Para calcular el valor del Estado 1: la suma de las recompensas si el agente comenzara en ese estado y luego siguiera la política greedy durante todos los pasos de tiempo.



Luego, para calcular $V(S_{t+1})$, necesitamos calcular el retorno comenzando desde ese estado S_{t+1} .



Estamos repitiendo el cálculo del valor de diferentes estados, lo cual puede resultar tedioso si necesitamos hacerlo para cada valor de estado o valor de estado-acción.

En lugar de calcular el retorno esperado para cada estado o para cada par estado-acción, podemos utilizar **la ecuación de Bellman**.

Ecuación de Bellman

La ecuación de Bellman es una ecuación **recursiva** que funciona de la siguiente manera: en lugar de comenzar desde el principio para cada estado y calcular el retorno, podemos considerar el valor de cualquier estado como:

La recompensa inmediata + el valor descontado del estado siguiente:

$$R_{t+1} + (\gamma * V(S_{t+1}))$$

The Bellman Equation

$$V_{\pi}(s) = \mathbf{E}_{\pi} [R_{t+1} + \gamma * V_{\pi}(S_{t+1}) | S_t = s]$$

Value of
state s

Expected value of
immediate reward

+ the discounted value of
next_state

If the agent
starts at state s

And uses the policy to
choose its actions for
all time steps



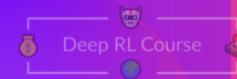
$V(S_t)$

R_{t+1}

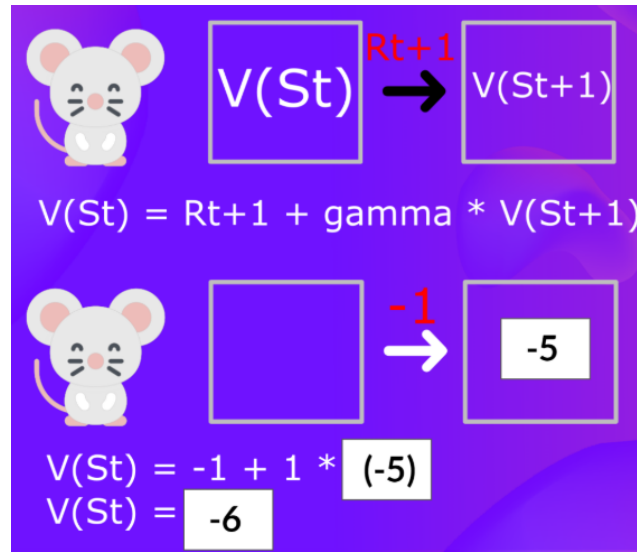
$V(S_{t+1})$



$$V(S_t) = R_{t+1} + \gamma * V(S_{t+1})$$



Para simplificar, aquí no aplicamos descuento, por lo que $\gamma = 1$.



En resumen, la idea de la ecuación de Bellman es que, en lugar de calcular cada valor como la suma del retorno esperado —lo cual es un proceso largo—, calculamos el valor como la suma de la **recompensa inmediata** + el **valor descontado del estado siguiente**.

Q-Learning

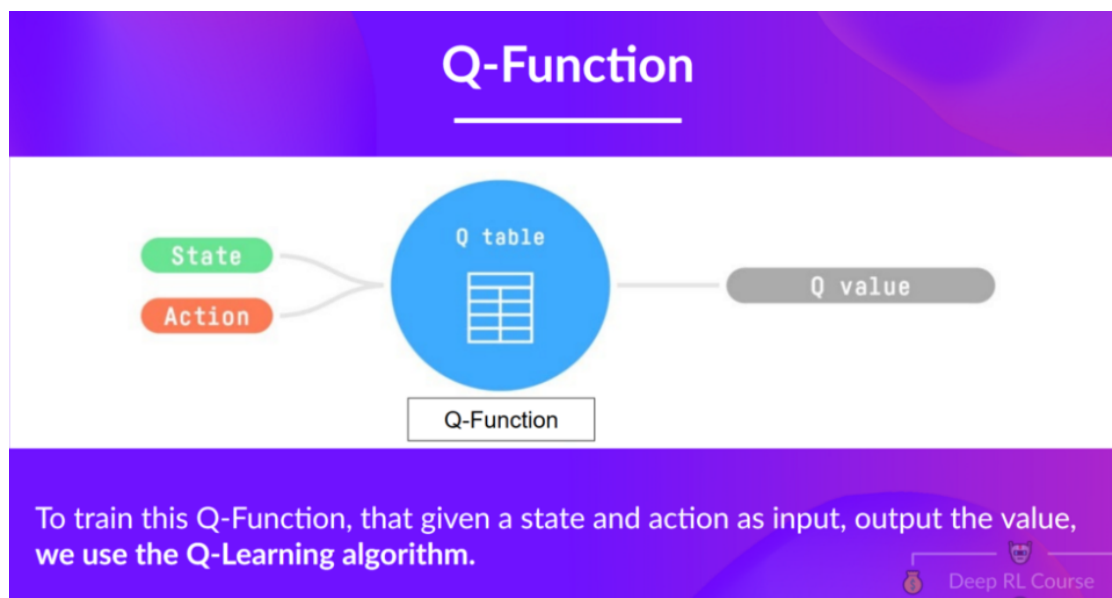
Método basado en valores y *off-policy* para entrenar su función de valor de acción:

- *Off-policy*: hablaremos de esto al final de esta unidad.
- *Método basado en valores*: encuentra la política óptima de manera indirecta, entrenando una función de valor o una función de valor de acción que nos indicará el valor de cada estado o de cada par estado-acción.

Actualiza su función de valor de acción en cada paso, en lugar de hacerlo al final del episodio.

Q-Learning es el algoritmo que utilizamos para entrenar nuestra función Q, una función de valor de acción que determina el valor de estar en un estado determinado y tomar una acción específica en ese estado.

Dado un estado y una acción, nuestra función Q devuelve un valor de estado-acción (también llamado **valor Q**).



Repasemos la diferencia entre valor y recompensa:

- El *valor de un estado*, o de un *par estado-acción*, es la recompensa acumulada esperada que obtiene nuestro agente si comienza en ese estado (o par estado-acción) y luego actúa de acuerdo con su política.
- La *recompensa* es la retroalimentación que recibe del entorno después de realizar una acción en un estado.

Internamente, nuestra función Q está representada mediante una **tabla Q** , una tabla en la que cada celda corresponde al valor de un par estado-acción.

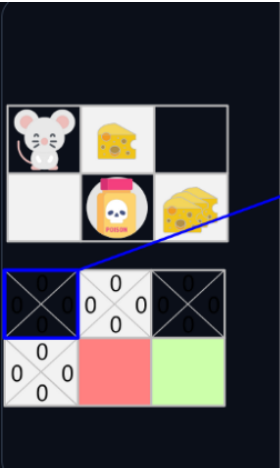


Diagram illustrating a grid world environment and its corresponding Q-table.

The environment is a 3x3 grid. The top row contains a mouse, a cheese, and an empty cell. The middle row contains an empty cell, a skull, and a cheese. The bottom row contains a 3x3 grid of cells with various symbols and numbers. A blue line connects the top-left cell of the bottom row to the first row of the Q-table.

The Q-table is a 6x5 matrix. The first column lists the states, and the subsequent four columns list the actions (Left, Right, Up, Down). All values in the table are 0, indicating that the Q-table is initialized.

	Actions			
	←	→	↑	↓
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

La tabla Q está inicializada. Por eso, todos los valores son $= 0$.

Esta tabla contiene, para cada estado y acción, los valores de estado-acción correspondientes. En este ejemplo sencillo, el estado está definido únicamente por la posición del ratón.

Por lo tanto, tenemos 2×3 filas en nuestra tabla Q, una fila por cada posición posible del ratón. En escenarios más complejos, el estado podría contener más información que la posición del agente.

Q-Learning es el algoritmo de RL que:

- Entrena una *función* Q (una función de valor de acción), que internamente es una tabla Q que contiene los valores de todos los pares estado-acción.
- Dado un estado y una acción, nuestra función Q buscará en su tabla Q el valor correspondiente.
- Cuando termina el entrenamiento, tenemos una función Q **óptima**, lo que significa que tenemos una tabla Q óptima.
- Y si tenemos una función Q óptima, tenemos una política óptima, ya que sabemos cuál es la mejor acción que tomar en cada estado.

Q-Learning

	←	→	↑	↓
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

→
Training

	←	→	↑	↓
	0	10.8	0	0
	0	9.9	0	-10
	0	0	0	10
	0	-10	0	0
	0	0	0	0
	0	0	0	0

Con el entrenamiento, nuestra tabla Q mejora, ya que gracias a ella podemos conocer el valor de cada par estado-acción.

Q-Learning

Algorithm 14: Sarsamax (Q-Learning)

Input: policy π , positive integer *num_episodes*, small positive fraction α , GLIE $\{\epsilon_i\}$

Output: value function Q ($\approx q_\pi$ if *num_episodes* is large enough)

Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)

```
for  $i \leftarrow 1$  to num_episodes do
     $\epsilon \leftarrow \epsilon_i$ 
    Observe  $S_0$ 
     $t \leftarrow 0$ 
    repeat
        Choose action  $A_t$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)
        Take action  $A_t$  and observe  $R_{t+1}, S_{t+1}$ 
         $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$ 
         $t \leftarrow t + 1$ 
    until  $S_t$  is terminal;
end
return  $Q$ 
```

↖ Step 1

Step 2

Step 3

Step 4

Q-Learning, Step 1

Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)

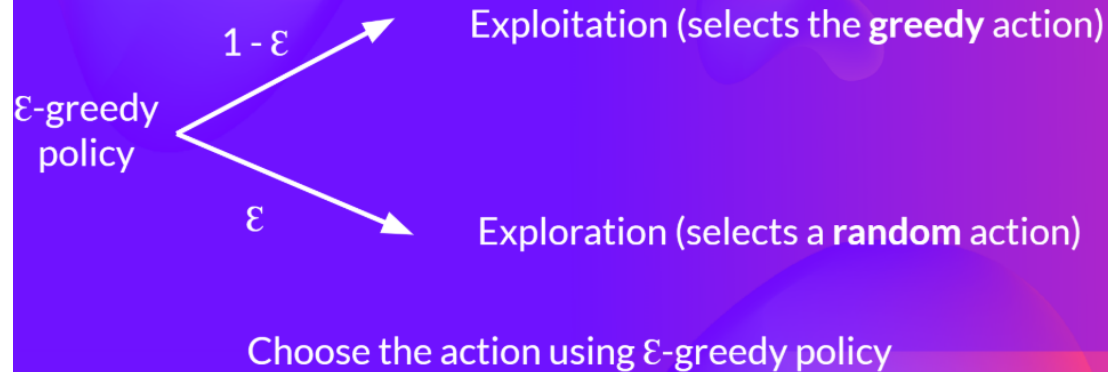
				
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0
	0	0	0	0

We initialize the Q-Table

Inicializamos Q arbitrariamente (ej. $Q(s, a) = 0$ para todo $s \in \mathcal{S}$ y $a \in \mathcal{A}(s)$, y $Q(\text{estado-terminal}, \cdot) = 0$): inicializamos la tabla Q .

Q-Learning, Step 2

Choose action A_t using policy derived from Q (e.g., ϵ -greedy)



Elegimos la acción A_t usando una política derivada de Q (ej. ϵ -greedy).

La estrategia **epsilon-greedy** es una política que gestiona el equilibrio entre exploración y explotación.

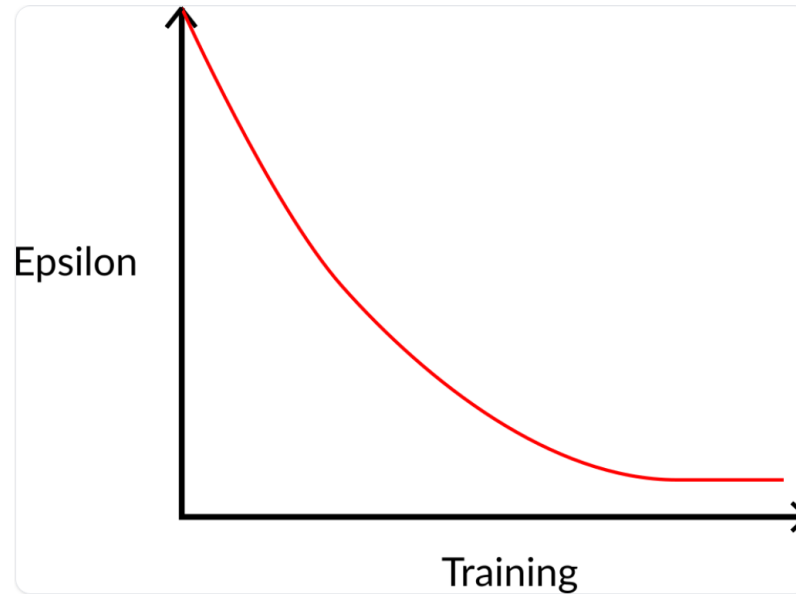
La idea es que, con un valor inicial de $\epsilon = 1.0$:

- Con una probabilidad de $1 - \epsilon$: hacemos **explotación** (es decir, nuestro agente selecciona la acción con el valor más alto para el par estado-acción).
- Con una probabilidad de ϵ : hacemos **exploración** (probando una acción aleatoria).

Al comienzo del entrenamiento,

la probabilidad de realizar exploración será muy alta

, ya que ϵ es muy alto, por lo que la mayor parte del tiempo exploraremos.



Ejecutamos la acción A_t y observamos R_{t+1}, S_{t+1} .

Q-Learning, Step 3

Take action A_t and observe R_{t+1}, S_{t+1}

Paso 4: actualizar $Q(S_t, A_t)$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

New
Q-value
estimation

Former
Q-value
estimation

Learning
Rate

Immediate
Reward

Discounted Estimate
optimal Q-value
of next state

Former
Q-value
estimation

TD Target

TD Error

Resumen

- Un agente de RL aprende su comportamiento por **prueba y error**, a partir de recompensas — no se le dice de antemano cuál es la política correcta.
- **Política** π : la función que decide qué acción tomar. Se puede entrenar *directamente* (métodos basados en políticas) o derivarse de una función de valor (métodos basados en valores).
- La **ecuación de Bellman** evita recalcular el retorno completo para cada estado: $V(s) = R_{t+1} + \gamma V(S_{t+1})$.
- **Q-Learning** entrena una tabla Q (valor de cada par estado-acción) actualizándola en *cada paso*, no al final del episodio.
- La política ϵ -**greedy** equilibra exploración (aleatoria) y explotación (la mejor acción según Q) — ϵ empieza alto y decae con el entrenamiento.
- Con una tabla Q óptima, la política óptima es inmediata: tomar en cada estado la acción de mayor valor Q.

Fin de la Clase 5

Próxima clase: Probabilidad, incertidumbre y redes bayesianas